

Architectural spec

This architectural specification details the transformation of a standard **Observe-Plan-Act** framework into a **Cognitive OODA** agent. As a Senior Architect, you'll recognize this as a move from simple **imperative automation** to a **stateful, goal-oriented system** capable of managing its own biases and learning from its failures.

I. Layered Memory Stack (The Orientation Engine)

OpenClaw's default state is "flat." This new specification introduces a **Tri-Tier Memory Architecture** that provides the contextual gravity needed for OODA.

Component	Logic Layer	Data Structure	Activity
Working Memory	Tier 1 (Short-term)	RAM / In-Context Buffer	Manages the "Active Attention." It handles turn-by-turn state, entity extraction, and provides the immediate context for tool outputs.
Episodic Memory	Tier 2 (Long-term)	Vector DB (e.g., ChromaDB)	The "Journal." It stores the raw historical logs (GitHub events, emails, chats) with semantic embeddings for similarity-based retrieval.
Semantic Memory	Tier 3 (Permanent)	KNOWLEDGE.json	The "Truth." It stores distilled, non-timestamped facts (e.g., "IEP Sensei uses FastAPI"). It is updated via the Archivist process.

Export to Sheets

II. The Multi-Model Reasoning Chain (OODA Loop)

This specification splits the "Agent Loop" into two specialized models to balance **reasoning depth** with **cost/latency**.

Phase A: The Triage (Observe & Orient)

- **Model:** Lightweight, High-Context (e.g., Gemini 1.5 Flash).
- **Input:** New environmental observations + Semantic Memory (Tier 3).
- **Activity (The Filter):** It performs **Salience Detection**. It asks: *"Does this new email from Brian actually conflict with Michael's stated 2026 priorities?"*

- **Output:** A **Situation Report (SITREP)** containing a Priority Score (1-10) and a Contextual Summary.

Phase B: The Strategy (The Decision Matrix)

This is the analytical engine that occurs **before** the final decision. It prevents the AI from jumping to the first logical plan it finds.

- **Activity 1: Hypothesis Generation:** The agent generates 2-4 distinct strategies (e.g., "Aggressive Fix," "Delegate Task," "Strategic Delay").
- **Activity 2: Multi-Variable Scoring:** Each strategy is scored against a **VALUATION_ENGINE** using a standard rubric:
 - **Alignment:** Does this match the **SOUL.md**?
 - **Efficiency:** Token cost vs. time saved.
 - **Risk:** Potential for breaking the build or missing an Austin meeting.
- **Activity 3: Optimization:** The agent selects the path that maximizes the **Global Objective Function** (e.g., maximizing IEP Sensei progress while maintaining Tesla factory commitments).

Phase C: The Executive (Decide & Act)

- **Model:** Large, Reasoning-Heavy (e.g., Claude 3.5 Sonnet / GPT-4o).
- **Input:** The SITREP + The Optimized Decision Matrix.
- **Activity:** It commits to the chosen strategy and generates a linear **Action Script** (the "Plan").
- **Output:** Tool calls and an **Expected Outcome** statement for the feedback loop.
-

III. Double-Loop Learning (The Adaptive Core)

Standard agents have "Single-Loop" learning (trying a different tool if one fails). This architecture implements **Double-Loop Learning**, allowing the agent to update its own rules.

1. The Archivist (Semantic Refinement)

- **Trigger:** Async cron (every 100 turns).
- **Activity:** Scans Tier 2 (Episodic) for repeated patterns.
- **Action:** If it notices Michael always ignores emails about "Marketing" but prioritizes "IEP Sensei Core," it updates **Semantic Memory** (Tier 3) to reflect this preference.

2. The Meta-Reviewer (Structural Refinement)

- **Trigger:** On "Critical Failure" or weekly review.
 - **Activity:** Analyzes why the **Expected Outcome** (from the Act phase) didn't match the **Actual Outcome**.
 - **Action:** If the failure was due to a rule in **SOUL.md** (e.g., "Always ask for permission before deleting files"), and that rule is hindering efficiency, it proposes a **Policy Update** to the user.
-

IV. The Hardest Part: Objective Weighting

The most difficult engineering hurdle is the **Transition from Heuristics to Ethics**.

- **The Breakdown:** We implement a **VALUATION_ENGINE** that takes the Path Score from the Decision Matrix and multiplies it by domain weights.
 - **Formula:** $V = \sum (S_i \cdot W_i)$, where S is the strategic score of a path and W is the domain weight (e.g., Fundraising = 1.0, Social = 0.4).
 - **Evolution:** Through Double-Loop learning, the agent adjusts W_i based on which decisions Michael approves or overrides over time.
-

Specification Summary

This architecture ensures that the agent doesn't just "execute a heartbeat"; it **re-evaluates its relationship with the world** every time it wakes up.

[A Technical Deep Dive into OpenClaw's Agentic Architecture](#) This video provides a technical walkthrough of how OpenClaw structures its gateway and agent sessions, which serves as the foundational "body" for the OODA-based brain we've specified.

Would you like me to generate the **PRIORITIES.json and **KNOWLEDGE.json** templates to start the Tier 3 implementation?**

Implementation spec

Phase 1 — Tri-Tier Memory Stack (Foundation)

This is the prerequisite for everything else. Without persistent context, the OODA loop has no Orientation.

New files to create:

- `packages/memory/` — new package. Three modules:
 - `working-memory.ts` — wraps the existing in-context session buffer (already partially exists in the Pi agent's session model). Add entity extraction on each turn to populate structured slots.
 - `episodic-memory.ts` — ChromaDB (or `better-sqlite3` with `sqlite-vec` if you want zero extra infrastructure) client. Handles `store(event)` and `retrieve(query, k)`. Triggered on every tool-output event the Gateway already emits.
 - `semantic-memory.ts` — read/write wrapper around `~/openclaw/workspace/KNOWLEDGE.json`. Exposes `getFacts()` and `upsertFact(key, value)` with JSON schema validation.

Modify:

- `src/agent/` (Pi agent runtime) — inject `getFacts()` output into the system prompt block currently assembled from `AGENTS.md` / `SOUL.md` / `TOOLS.md`. This is additive and low-risk.

`~/openclaw/workspace/KNOWLEDGE.json` — create the template the spec hints at.

Suggested schema:

```
{ "facts": { "stack": "FastAPI", "timezone": "US/Central" }, "updated_at": "..." }
```

-

OpenClaw hooks to use: The existing `agents.defaults.workspace` injection pipeline and the `sessions_history` tool already give you the raw material for episodic storage — pipe those events into ChromaDB writes.

Phase 2 — Multi-Model Reasoning Chain (OODA Brain)

This is the biggest structural change. The single Pi agent RPC call splits into two sequential model invocations before executing any tools.

New file: `packages/ooda/triage.ts`

The Triage phase takes the incoming observation plus Tier 3 facts and calls a lightweight model (Gemini Flash or `claude-haiku-4-5`). It outputs a typed `SITREP` object:

```
interface SITREP {  
  priority: 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10;  
  summary: string;  
  conflictsDetected: string[];  
  relevantFacts: string[];  
}
```

New file: `packages/ooda/strategy.ts`

The Decision Matrix phase generates 2–4 candidate strategies and scores each against your `VALUATION_ENGINE`:

```
interface Strategy {  
  label: string; // e.g. "Aggressive Fix"  
  alignmentScore: number;  
  efficiencyScore: number;  
  riskScore: number;  
  weightedTotal: number; //  $V = \sum(S_i \times W_i)$   
}
```

Domain weights W_i are read from a new `PRIORITIES.json` file in the workspace.

New file: `~/openclaw/workspace/PRIORITIES.json`

```
{  
  "domains": {  
    "core_project": 1.0,  
    "communication": 0.7,  
    "social": 0.4  
  }  
}
```

Modify: `src/agent/pi.ts` (or equivalent agent loop entrypoint) — wrap the existing single LLM call so it runs `triage` → `strategy` → `executive` in sequence. The Executive phase gets the SITREP + winning strategy as structured context prepended to its system prompt.

OpenClaw hooks to use: The existing model failover system (`model-failover` docs) handles the lightweight-vs-heavy model routing. The `openclaw.json` config already supports per-agent model assignment — use `agents.defaults.model` for the Executive and introduce `agents.defaults.triageModel` for the Triage pass.

Phase 3 — Double-Loop Learning (The Adaptive Core)

Both learners are async and should run outside the hot path. OpenClaw already has cron and webhook infrastructure — use it directly.

New file: `packages/ooda/archivist.ts`

Registers itself as a cron job via the Gateway's existing cron API (`automation/cron-jobs`). Every 100 turns it queries Tier 2 (episodic), runs a summarization pass to extract stable preferences, and calls `semantic-memory.upsertFact()` to persist them into Tier 3.

New file: `packages/ooda/meta-reviewer.ts`

Triggered on two conditions: (1) a new `criticalFailure` event type you'll add to the Gateway's event bus, and (2) a weekly cron. It diffs `expectedOutcome` (stored by the Executive in each turn's metadata) against `actualOutcome` (detected from follow-up turns). If a SOUL.md rule is implicated, it emits a `policyProposal` that surfaces in the Control UI for manual approval — **never auto-updates SOUL.md without the user seeing it**.

Key implementation note: Store `expectedOutcome` and `actualOutcome` in each session's metadata. The existing `sessions.patch` WS method already persists arbitrary session metadata alongside `thinkingLevel`, `model`, etc. — just extend that schema.

Phase 4 — Objective Weighting / VALUATION_ENGINE

This is relatively small code but has the highest long-term leverage.

New file: `packages/ooda/valuation-engine.ts`

Implements $V = \sum(S_i \times W_i)$ with the domain weights from `PRIORITIES.json`. Exposes a `score(strategies: Strategy[]): Strategy` function that returns the winning path.

Modify: `packages/ooda/strategy.ts` — call the valuation engine after generating hypotheses.

Modify: `meta-reviewer.ts` — when a user overrides or approves a strategy, record the domain context and adjust the relevant W_i in `PRIORITIES.json`. This closes the learning loop on objective weights over time.

Suggested PR Sequence

Start small and stay shippable:

1. **PR 1:** Tier 3 only — `KNOWLEDGE.json` template + `semantic-memory.ts` + injection into the system prompt. Zero risk, immediately useful.
 2. **PR 2:** Tier 2 — `episodic-memory.ts` + passive event logging into ChromaDB. Read-only for now.
 3. **PR 3:** Triage phase — `triage.ts` + `SITREP` type + lightweight model call wired before the Executive. Feature-flagged off by default.
 4. **PR 4:** Decision Matrix + `VALUATION_ENGINE`. Feature-flagged.
 5. **PR 5:** Archivist cron. Safe since it's purely read → write to Tier 3.
 6. **PR 6:** Meta-Reviewer + outcome tracking. Requires the new `criticalFailure` event type.
 7. **PR 7:** Remove feature flags, wire `PRIORITIES.json` weight adjustment.
-

The biggest watch-out is Phase 2: doubling the model call count per turn will increase latency noticeably. I'd gate the full Triage → Strategy chain behind a `thinkingLevel >= "medium"` check — for quick conversational turns, skip straight to the Executive. That preserves responsiveness for simple interactions while unlocking full OODA for complex ones where it matters.

A few design decisions worth calling out:

KNOWLEDGE.json — the `_archivist_log` array at the bottom is append-only and written by the Archivist cron, never by the Executive. This gives you a cheap audit trail so you can debug *why* a fact got distilled without needing to dig through episodic logs. The `preferences.never_do` and `always_ask_before` arrays are the most important fields to

populate early — they're the ones that prevent the agent from doing something irreversible during the Act phase.

PRIORITIES.json — a few things to note:

- The `approval_count` / `override_count` fields on each domain entry are what the Meta-Reviewer reads to nudge weights over time. The formula is simple: $\text{new_weight} = \text{current_weight} * (1 + (\text{approvals} - \text{overrides}) * 0.05)$, clamped to $[0.1, 1.0]$. You'd tune that coefficient.
- The `thresholds` block is the feature-flag mechanism from the implementation plan — `min_thinking_level_for_full_ooda: "medium"` means quick conversational turns bypass the full Triage → Strategy chain entirely.
- The `scoring_rubric` weights (alignment 40%, efficiency 35%, risk 25%) are a starting point. Risk is intentionally the smallest weight because overly risk-averse agents become useless — but you may want to bump it during the early trust-building period.

Both files should land at `~/.openclaw/workspace/` so they're picked up by the existing workspace injection pipeline alongside `SOUL.md`.

Knowledge.json

```

{
  "$schema": "https://openclaw.ai/schemas/knowledge/v1",
  "_meta": {
    "version": 1,
    "updated_at": "2026-03-16T00:00:00Z",
    "updated_by": "archivist",
    "turn_count_at_last_update": 0,
    "description": "Tier 3 Semantic Memory — distilled, non-timestamped facts about the user
and their world. Updated by the Archivist process; never written by the Executive model
directly."
  },

  "identity": {
    "name": "",
    "timezone": "",
    "location_primary": "",
    "language_primary": "en",
    "communication_style": ""
  },

  "stack": {
    "description": "Known technical stack facts. Keys are project or domain names; values are
free-form strings.",
    "entries": {
      "example_project": "FastAPI + PostgreSQL + React"
    }
  },

  "projects": {
    "description": "Active projects with their current status and key constraints.",
    "entries": {
      "example_project": {
        "status": "active",
        "priority_domain": "core_project",
        "key_constraint": "Must ship before end of Q2",
        "notes": ""
      }
    }
  },

  "people": {
    "description": "Key people the agent interacts with on behalf of the user.",
    "entries": {
      "example_contact": {

```

```

    "role": "",
    "relationship": "",
    "communication_preference": "",
    "notes": ""
  }
}
},

"preferences": {
  "description": "Stable behavioral preferences inferred by the Archivist from episodic
patterns.",
  "always_ask_before": [],
  "never_do": [],
  "prefers_async_over_sync": false,
  "prefers_delegation_over_diy": false,
  "response_length": "concise",
  "notes": ""
},

"commitments": {
  "description": "Recurring obligations that must be protected during scheduling and task
planning.",
  "entries": [
    {
      "label": "example_commitment",
      "recurrence": "weekly",
      "day": "Monday",
      "time": "10:00",
      "timezone": "",
      "blocking": true
    }
  ]
},

"domain_context": {
  "description": "Free-form domain knowledge the agent should assume without being told each
turn.",
  "entries": {}
},

"_archivist_log": {
  "description": "Readonly audit trail of Archivist update reasons. Appended, never edited.",
  "entries": []
}

```

}

Priorities.json

```

{
  "$schema": "https://openclaw.ai/schemas/priorities/v1",
  "_meta": {
    "version": 1,
    "updated_at": "2026-03-16T00:00:00Z",
    "updated_by": "user",
    "description": "Domain weights ( $W_i$ ) for the VALUATION_ENGINE. Updated by the
Meta-Reviewer based on approval/override history. All weights normalized 0.0–1.0. A weight of
1.0 means this domain is never deprioritized."
  },

  "domains": {
    "description": "Each key is a domain name. The agent uses these weights when scoring
candidate strategies via  $V = \sum(S_i \times W_i)$ . Add, rename, or remove domains freely — the engine
will pick them up on next boot.",
    "entries": {
      "core_project": {
        "weight": 1.0,
        "description": "Primary product or project work",
        "examples": ["feature development", "bug fixes", "architecture decisions"],
        "approval_count": 0,
        "override_count": 0
      },
      "fundraising": {
        "weight": 1.0,
        "description": "Investor relations, fundraising, and strategic partnerships",
        "examples": ["pitch prep", "investor updates", "term sheet review"],
        "approval_count": 0,
        "override_count": 0
      },
      "operations": {
        "weight": 0.8,
        "description": "Day-to-day operational work: hiring, finance, legal",
        "examples": ["contracts", "payroll", "vendor management"],
        "approval_count": 0,
        "override_count": 0
      },
      "communication": {
        "weight": 0.7,
        "description": "External comms: customer emails, press, community",
        "examples": ["support tickets", "newsletter", "social media"],
        "approval_count": 0,
        "override_count": 0
      }
    }
  }
}

```

```

"research": {
  "weight": 0.6,
  "description": "Learning, reading, exploratory investigation",
  "examples": ["reading papers", "competitor analysis", "tech spikes"],
  "approval_count": 0,
  "override_count": 0
},
"maintenance": {
  "weight": 0.5,
  "description": "Housekeeping: inbox zero, file organization, dependency updates",
  "examples": ["email triage", "repo cleanup", "calendar hygiene"],
  "approval_count": 0,
  "override_count": 0
},
"social": {
  "weight": 0.4,
  "description": "Low-stakes social interactions and informal networking",
  "examples": ["casual Slack threads", "conference chatter"],
  "approval_count": 0,
  "override_count": 0
}
}
},

"strategy_labels": {
  "description": "Named strategy archetypes the Hypothesis Generator is instructed to consider.
Extend this list to bias the agent toward specific styles of response.",
  "entries": [
    {
      "label": "aggressive_fix",
      "description": "Act immediately with full effort and available resources"
    },
    {
      "label": "delegate_task",
      "description": "Route to another person or agent; track to completion"
    },
    {
      "label": "strategic_delay",
      "description": "Defer until a better moment; set a reminder and context note"
    },
    {
      "label": "minimal_viable_action",
      "description": "Do the smallest thing that unblocks progress"
    }
  ]
}

```

```

]
},

"scoring_rubric": {
  "description": "The three axes scored for each strategy candidate. Weights here control their
relative importance inside the valuation formula.",
  "alignment": {
    "weight": 0.40,
    "description": "How well the strategy matches the active domain's goals and the user's
stated preferences in SOUL.md"
  },
  "efficiency": {
    "weight": 0.35,
    "description": "Token cost and time spent relative to expected value delivered"
  },
  "risk": {
    "weight": 0.25,
    "description": "Potential for irreversible side-effects, missed commitments, or build breakage.
Lower risk = higher score."
  }
},

"thresholds": {
  "description": "Control when the full Triage → Strategy → Executive chain fires vs. the fast
path.",
  "min_priority_for_full_ooda": 5,
  "min_thinking_level_for_full_ooda": "medium",
  "critical_failure_score_floor": 3,
  "archivist_turn_interval": 100,
  "meta_reviewer_weekly_enabled": true
},

"_weight_adjustment_log": {
  "description": "Readonly audit trail written by the Meta-Reviewer when it adjusts domain
weights based on approval/override patterns. Appended, never edited.",
  "entries": []
}
}

```

Fixes

Fixes

Architectural Spec

- **Section II, Phase C** — add the prompt contract for Triage output (blocker #4). This is where the SITREP interface belongs, not just in the implementation spec. The model I/O contract is architecture, not implementation detail.
 - **Section III, item 2 (Meta-Reviewer)** — rewrite the `actualOutcome` paragraph (blocker #2) and define the `criticalFailure` event schema (blocker #3). Both are currently one-liners that hand-wave the hardest engineering problem in the spec.
 - **Section IV** — add the weight adjustment formula (ambiguity #7). It's currently only in our chat transcript.
-

Implementation Spec

- **Phase 1, "New files to create"** — add the TypeScript interfaces for `KnowledgeFile` and `PrioritiesFile` as the canonical type definitions (blocker #1). The JSON templates are downstream of these.
 - **Phase 1, "Modify" bullet** — specify the injection order in the system prompt (ambiguity #10).
 - **Phase 2, "New file: packages/ooda/triage.ts"** — add the skeleton prompt template and specify output format (JSON/YAML/natural language) (blocker #4 again — the implementation spec needs the detail the arch spec introduces).
 - **Phase 2, "New file: ~/.openclaw/workspace/PRIORITIES.json"** — remove the two-domain stub and replace with a pointer to the full template, or the discrepancy between the stub and the real file will confuse dev (minor but real).
 - **Phase 3, "New file: packages/ooda/archivist.ts"** — specify where the turn counter lives and what happens on gateway restart (ambiguity #9).
 - **Phase 3, "New file: packages/ooda/meta-reviewer.ts"** — replace "surfaces in the Control UI" with a concrete delivery mechanism (ambiguity #11), and add the `policyProposal` payload type.
 - **Phase 4, "New file: packages/ooda/valuation-engine.ts"** — add the startup assertion requirement for rubric weights summing to 1.0 (blocker #6).
 - **"Suggested PR Sequence"** — add a test file alongside each new package (ambiguity #12), and add a decision on ChromaDB vs `sqlite-vec` before PR 2 (ambiguity #8). That decision gates the entire PR sequence.
-

Both schema templates (KNOWLEDGE.json, PRIORITIES.json)

- Both files — fix the `$schema` URL (blocker #5). Either point to a local `schemas/` path in the repo or remove the field until the schema is published.

- `KNOWLEDGE.json` — flatten `stack`, `projects`, `people` so `description` moves to a comment or `_meta` block rather than sitting as a peer to data keys (blocker #1, structural half).
 - `PRIORITIES.json` — flatten `domains` so `entries` is removed and domain keys sit directly under `domains`, matching OpenClaw's config conventions (blocker #1, structural half).
-

The heaviest rewrite is the Implementation Spec Phase 3 Meta-Reviewer section and the two schema files. Everything else is additive — a paragraph or a type definition dropped into an existing section. If you want to triage by effort, fix the schemas and the Meta-Reviewer section first; those are the ones most likely to cause silent integration failures.